1. dijkstra’s alg

```
#include <stdio.h>
#define INF 999  // Represents infinity (no direct path)

int main()
{
   // Declare arrays and variables
   int cost[10][10], dist[10], vis[10] = {0};
   int pre[10], path[10], n, s, t;
   int i, j, u, min, k = 0;

   // Read number of nodes
   printf("Enter number of nodes: ");
   scanf("%d", &n);

   // Read adjacency matrix
   printf("Enter adjacency matrix:\n");
   for (i = 1; i <= n; i++)
      for (j = 1; j <= n; j++)
      {
         scanf("%d", &cost[i][j]);

         // Zero means no edge between different nodes
         if (i != j && cost[i][j] == 0)
            cost[i][j] = INF;
      }
```

```
// Read source and destination nodes
printf("Enter source and target: ");
scanf("%d%d", &s, &t);

// Initialize distances and predecessors
for (i = 1; i <= n; i++)
{
   dist[i] = INF;  // Initially, distance is infinity
   pre[i] = -1;    // No previous node
}

dist[s] = 0;  // Distance from source to itself is zero

// Repeat to find the shortest distances
for (i = 1; i <= n; i++)
{
   min = INF;
   u = -1;

   // Find the unvisited node with minimum distance
   for (j = 1; j <= n; j++)
      if (!vis[j] && dist[j] < min)
         min = dist[j], u = j;

   // Stop if no reachable unvisited node exists
   if (u == -1)
      break;
```

```
    vis[u] = 1;  // Mark node as visited

    // Update distances to neighbouring nodes
    for (j = 1; j <= n; j++)
        if (!vis[j] && cost[u][j] != INF &&
            dist[u] + cost[u][j] < dist[j])
        {
            dist[j] = dist[u] + cost[u][j];
            pre[j] = u;  // Store previous node
        }
}

// Check whether destination is reachable
if (dist[t] == INF)
{
    printf("No path exists.\n");
    return 0;
}

// Trace the shortest path backwards
for (u = t; u != -1; u = pre[u])
    path[k++] = u;

// Print path in the correct order
printf("Shortest Path: ");
for (i = k - 1; i >= 0; i--)
    printf("%d%s", path[i], i ? " -> " : "");
```

```
    // Print total shortest distance
    printf("\nShortest distance = %d\n", dist[t]);

    return 0;
}
```

2. crc

```
#include <stdio.h>
#include <string.h>

char data[100], check_value[100], gen_poly[20];
int data_length, i, j, n;

// Perform XOR division
void XOR()
{
    for (j = 1; j < n; j++)
        check_value[j] =
            (check_value[j] == gen_poly[j]) ? '0' : '1';
}

// Calculate CRC remainder
void crc()
{
    // Start with the first n bits
    for (i = 0; i < n; i++)
        check_value[i] = data[i];
```

```
    // Process the data bits one by one
    do
    {
        // XOR when the first bit is 1
        if (check_value[0] == '1')
            XOR();

        // Shift the bits left
        for (j = 0; j < n - 1; j++)
            check_value[j] = check_value[j + 1];

        // Bring down the next data bit
        check_value[j] = data[i++];
    } while (i <= data_length + n - 1);

    // Terminate the remainder string
    check_value[n - 1] = '\0';
}

// Check the received data for errors
void receiver()
{
    printf("\nEnter the received data: ");
    scanf("%99s", data);

    data_length = strlen(data);

    // Calculate the remainder of received data
```

```
    crc();

    // If any remainder bit is 1, an error is detected
    for (i = 0; i < n - 1; i++)
    {
        if (check_value[i] == '1')
            break;
    }

    if (i < n - 1)
        printf("Error detected in received data!\n");
    else
        printf("No error detected in received data.\n");
}

int main()
{
    printf("Enter data to be transmitted: ");
    scanf("%99s", data);

    printf("Enter the generating polynomial: ");
    scanf("%19s", gen_poly);

    n = strlen(gen_poly);
    data_length = strlen(data);

    // Append n-1 zeros to the original data
    for (i = data_length; i < data_length + n - 1; i++)
```

```
    data[i] = '0';

  data[data_length + n - 1] = '\0';

  printf("\nData after appending zeros: %s\n", data);

  // Calculate CRC remainder
  crc();

  printf("CRC (Check Value): %s\n", check_value);

  // Append the CRC remainder to the original data
  for (i = 0; i < n - 1; i++)
    data[data_length + i] = check_value[i];

  data[data_length + n - 1] = '\0';

  printf("Final data to be transmitted: %s\n", data);

  // Receiver checks the transmitted data
  receiver();

  return 0;
}
```

3. bit stuffing

```
#include <stdio.h>
```

```
// Function to perform Bit Stuffing
void bitStuffing(int N, int arr[])
{
   // Array to store the stuffed bit sequence
   int brr[100];

   // i = input index, j = output index, count = consecutive 1s
   int i, j = 0, count = 0;

   // Traverse each bit in the input array
   for (i = 0; i < N; i++)
   {
      // Copy the current bit into the output array
      brr[j++] = arr[i];

      // Count consecutive 1s
      if (arr[i] == 1)
         count++;
      else
         count = 0;  // Reset count if the bit is 0

      // If five consecutive 1s are found, insert a 0
      if (count == 5)
      {
         brr[j++] = 0;  // Insert the stuffed bit
         count = 0;     // Reset the counter
      }
   }
```

```
    // Print the stuffed bit sequence
    printf("Stuffed Bit Sequence: ");

    // Print all bits in the output array
    for (i = 0; i < j; i++)
        printf("%d", brr[i]);
}

// Main function
int main()
{
    // Number of bits in the input sequence
    int N = 6;

    // Original bit sequence
    int arr[] = {1, 1, 1, 1, 1, 1};

    // Call the Bit Stuffing function
    bitStuffing(N, arr);

    return 0;  // End the program
}
```

4. character stuffing

```
#include <stdio.h>
#include <string.h>

int main()
```

```
{
   char a[100];           // Original data
   char fs[205] = "";      // Stuffed frame
   char sd, ed;           // Starting and ending delimiters
   int i, j = 0, len;

   // Input original data
   printf("Enter characters to be stuffed: ");
   scanf("%99s", a);

   // Input starting and ending delimiters
   printf("Enter starting delimiter: ");
   scanf(" %c", &sd);

   printf("Enter ending delimiter: ");
   scanf(" %c", &ed);

   // Delimiters must be different
   if (sd == ed)
   {
      printf("Delimiters must be different!\n");
      return 1;
   }

   // Add starting delimiter
   fs[j++] = sd;

   // Traverse the original data
```

```
for (i = 0; a[i] != '\0'; i++)
{
   // Copy the current character
   fs[j++] = a[i];

   // Duplicate the character if it matches a delimiter
   if (a[i] == sd || a[i] == ed)
      fs[j++] = a[i];
}

// Add ending delimiter
fs[j++] = ed;
fs[j] = '\0';          // End the string

// Print the stuffed frame
printf("\nAfter Stuffing: %s", fs);

// Character De-Stuffing
len = strlen(fs);
j = 0;

// Skip starting and ending delimiters
for (i = 1; i < len - 1; i++)
{
   // Copy the data character
   a[j++] = fs[i];

   // Skip the extra copy of a stuffed delimiter
```

```
        if (fs[i] == sd || fs[i] == ed)
            i++;
    }

    a[j] = '\0';            // End the de-stuffed string

    // Print the original data
    printf("\nAfter De-Stuffing: %s\n", a);

    return 0;
}
```